

Data Structures

Mushtari Sadia



Introduction

Faculty Name: Mushtari Sadia (MIS)

Consultation Hours:

Wednesday: 9:30AM-1:45PM

Thursday: 9:30AM-10:40AM

Room: **UB-80601**

Join Slack Channel:

https://join.slack.com/t/cse220summer2023/shared_invite/zt-1wrcesdkw-7bGTJMs62ihUWRlq4RVIA
[A](#)

Resources link: tiny.cc/cse220mis

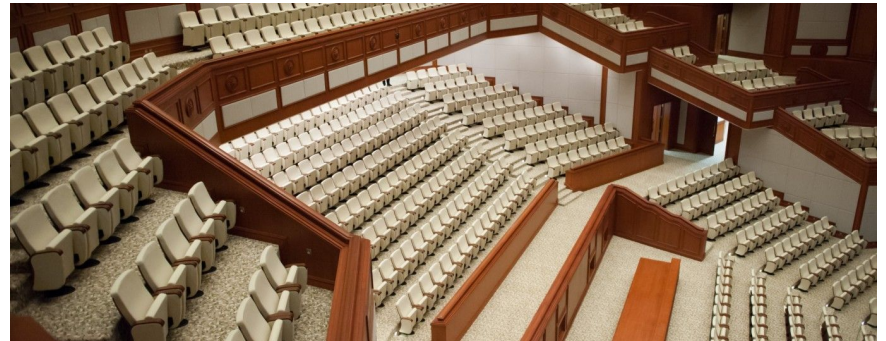


Multidimensional Arrays and Linked List

Week 2



Multi-Dimensional Array



Multi-Dimensional Array

1D Array

3	2
---	---

2D Array

1	0	1
3	4	1

3D Array

1	7	9
5	9	3
7	9	9

Multi-Dimensional Array

```
#Let's create a one-dimensional array of five elements  
array_1D = [None]*5
```

```
#Let's create a two-dimensional array of size 4x5  
array_2D = [None]*4  
for i in range(len(array_2D)):  
    array_2D[i] = [None]*5
```

```
#Let's see how to access elements
```

```
print(array_1D[3]) #We're accessing the 3rd element of the array  
print(array_2D[2][3]) #check the 3rd element of the 2nd row
```

3rd element of the array

3rd element of 2nd row

How are multi-dimensional arrays stored?

The following diagram shows the difference between these two approaches for a 2D matrix of 3×3 dimensions. (By the way, row is the vertical and column is the horizontal axis in 2D. You should already know that. Here Dimension 0 represents the row and Dimension 1 represents the column.)

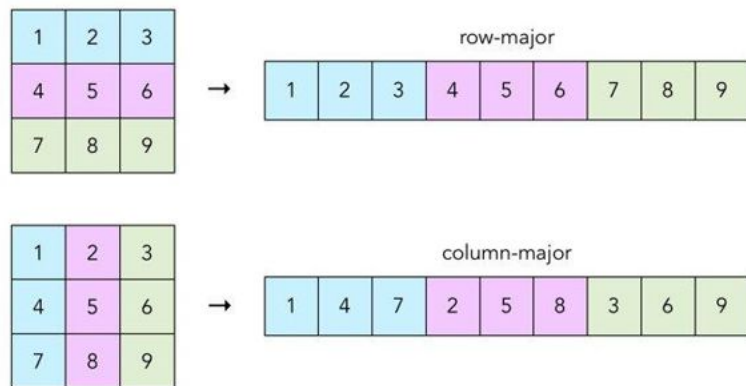


Figure 1: Two Different Ways of Storing Multidimensional Arrays in a Linear Format

How are multi-dimensional arrays stored?

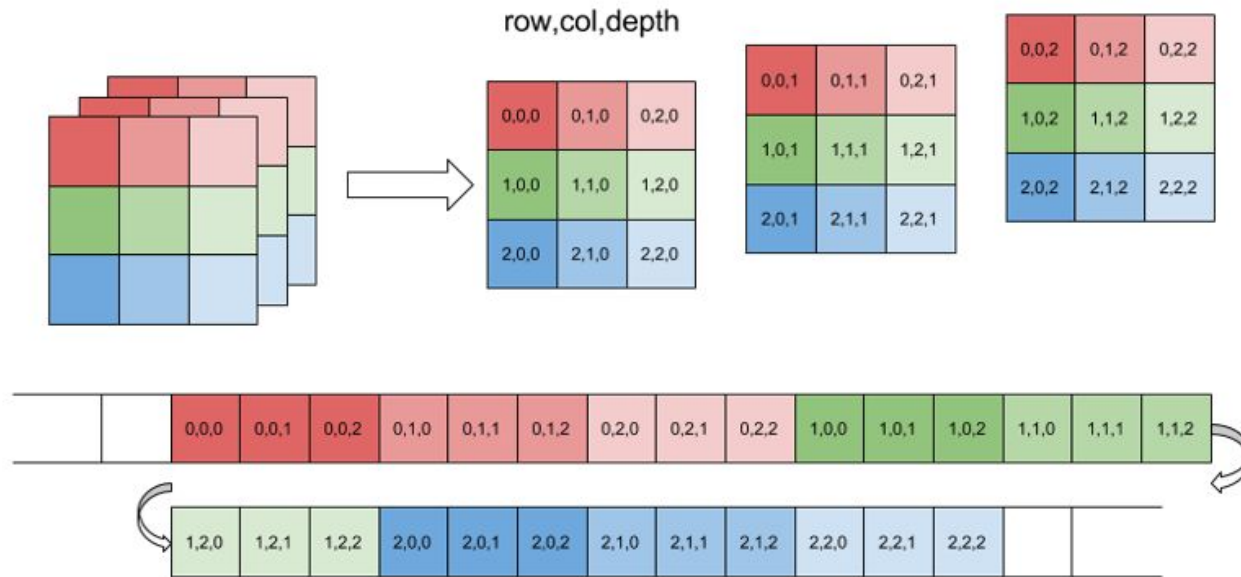


Figure 2: Row Major Ordering of Indexes of a 3D Array

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box[w][x][y][z]*.

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + y \times P + z$$

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box[w][x][y][z]*.

$$\text{Linear Index} = \underline{w \times (N \times O \times P)} + x \times (O \times P) + y \times P + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box* $[w][x][y][z]$.

$$\text{Linear Index} = w \times (N \times O \times P) + \underline{x \times (O \times P)} + y \times P + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box*[*w*][*x*][*y*][*z*].

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + \underline{y \times P} + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box*[*w*][*x*][*y*][*z*].

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + y \times P + \underline{z}$$

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions 4x3x5x6, named *box*

element at the index you are interested is the `box[2][1][4][3]`

Linear Index = ??

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

Linear Index = 111 -> `box[x][y][z]`

Find x,y,z

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\text{Linear Index} = x * (4*8) + y*8 + z$$

$$= \mathbf{111}$$

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned} \text{Linear Index} &= x * (4*8) + y*8 + z && \text{where } 0 \leq x \leq 3, 0 \leq y \leq 3 \text{ and } 0 \leq z \leq 7 \\ &= \mathbf{111} \end{aligned}$$

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned}\text{Linear Index} &= x * (4*8) + y*8 + z \\ &= 111\end{aligned}$$

where $0 \leq x \leq 3$, $0 \leq y \leq 3$ and $0 \leq z \leq 7$

Remember that indexing starts from 0 in each dimension!

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned}\text{Linear Index} &= x * (4*8) + y*8 + z \\ &= 111\end{aligned}$$

where $0 \leq x \leq 3$, $0 \leq y \leq 3$ and $0 \leq z \leq 7$

Remember that indexing starts from 0 in each dimension!

$$4 \times 8 =$$

		0	0	3
3	2	1	1	1
	-	0		
		1	1	
	-		0	
		1	1	1
	-		9	6
		1	5	

Quotient

$x=3$

Remainder

Linear Index -> Multidimensional Array Index

Following the process we get,

$$X = 111 // (4 * 8) = 3 \text{ and } 111 \% (4 * 8) = 15$$

$$Y = 15 // 8 = 1 \text{ and } 15 \% 8 = 7$$

$$Z = 7$$

So, the dimension of linear index 111 is [3][1][7].

Linear Index -> Multidimensional Array Index

Practice

Find the dimension of 96, 107, and 60 of the linear index.

Array Initialization with NumPy

▼ Learning To Use Numpy

```
✓ [1] 1 import numpy as np  
0s
```

```
✓ [5] 1 arr = np.array([1, 2, 3, 4, 5])  
0s 2  
3 print(arr)
```

```
[1 2 3 4 5]
```

```
✓ [2] 1 array = np.zeros(5)  
0s 2 print(array)
```

```
[0. 0. 0. 0. 0.]
```

Array Initialization with NumPy

✓
0s

```
[3] 1 array[2] = 5  
    2 print(array)
```

[0. 0. 5. 0. 0.]

```
[4] 1 for i in range(len(array)):  
    2 |   array[i] = i+1  
    3  
    4 print(array)
```

[1. 2. 3. 4. 5.]

Array Initialization with NumPy

✓ 0s 1 print(array[1])

2.0

✗ 0s [7] 1 print(array[11])

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-7-92562eb7d277> in <cell line: 1>()  
----> 1 print(array[11])
```

IndexError: index 11 is out of bounds for axis 0 with size 5

[SEARCH STACK OVERFLOW](#)

✓ 0s [8] 1 print(array[0])

1.0

✗ 0s [10] 1 print(array[5])

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-10-7de47b3bb65c> in <cell line: 1>()  
----> 1 print(array[5])
```

IndexError: index 5 is out of bounds for axis 0 with size 5

Multi-Dimensional Array Initialization with NumPy

▼ Multidimensional Array Initialization

```
[ ] 1 # Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:  
2 arr = np.array([[1, 2, 3], [4, 5, 6]])  
3  
4 print(arr)
```

```
[[1 2 3]  
 [4 5 6]]
```

```
[12] 1 #Create a 2D array with the dimensions 2x3  
2 arr = np.zeros((2,3))  
3 print(arr)
```

```
[[0. 0. 0.]  
 [0. 0. 0.]]
```

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1  #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3  arr = np.zeros((2,3,5))  
        4  print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]]  
  
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]])
```

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1 #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3 arr = np.zeros((2,3,5))  
        4 print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]
```

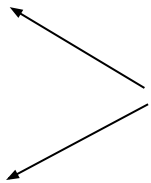
```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]
```

Two lists

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1 #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3 arr = np.zeros((2,3,5))  
        4 print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]]  
  
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]]]
```

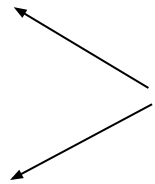


Two lists **having**
three lists each

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1  #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3  arr = np.zeros((2,3,5))  
        4  print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
   
  [[0. 0. 0. 0. 0.]  
   [0. 0. 0. 0. 0.]  
   [0. 0. 0. 0. 0.]]]
```



Two lists having three
lists each
**having 5 elements
each**

Iterating over a Multidimensional Array

```
✓ [14] 1 array_2D = np.zeros((3,3))  
    2  
    3 print(array_2D.shape)  
  
(3, 3)
```

```
✓ [19] 1 print(array_3D.shape)  
0s  
(4, 5, 6)
```

```
✓ [20] 1 print(array_3D.shape[0])  
0s  
4
```

```
✓ [21] 1 print(array_3D.shape[1])  
0s  
5
```

```
✓ [22] 1 print(array_3D.shape[2])  
0s  
6
```

Iterating over a Multidimensional Array

✓
0s



```
1  for i in range(array_2D.shape[0]):  
2      for j in range(array_2D.shape[1]):  
3          array_2D[i][j] = 9  
4  
5  print(array_2D)
```



```
[[9. 9. 9.]  
 [9. 9. 9.]  
 [9. 9. 9.]]
```


Iterating over a Multidimensional Array

✓
0s



```
1 array_3D = np.zeros((4,5,6))
2
3 for i in range(array_3D.shape[0]):
4     for j in range(array_3D.shape[1]):
5         for k in range(array_3D.shape[2]):
6             array_3D[i][j][k] = i*100+j*10+k
7
8 print(array_3D)
```

Iterating over a Multidimensional Array

```
✓ 0s [▶] 1 array_3D = np.zeros((4,5,6))
2
3 for i in range(array_3D.shape[0]):
4     for j in range(array_3D.shape[1]):
5         for k in range(array_3D.shape[2]):
6             array_3D[i][j][k] = i*100+j*10+k
7
8 print(array_3D)
```

```
▶ [[ 0.  1.  2.  3.  4.  5.]
   [10. 11. 12. 13. 14. 15.]
   [20. 21. 22. 23. 24. 25.]
   [30. 31. 32. 33. 34. 35.]
   [40. 41. 42. 43. 44. 45.]]
```

```
[[100. 101. 102. 103. 104. 105.]
 [110. 111. 112. 113. 114. 115.]
 [120. 121. 122. 123. 124. 125.]
 [130. 131. 132. 133. 134. 135.]
 [140. 141. 142. 143. 144. 145.]]
```

```
[[200. 201. 202. 203. 204. 205.]
 [210. 211. 212. 213. 214. 215.]
 [220. 221. 222. 223. 224. 225.]
 [230. 231. 232. 233. 234. 235.]
 [240. 241. 242. 243. 244. 245.]]
```

```
[[300. 301. 302. 303. 304. 305.]
 [310. 311. 312. 313. 314. 315.]
 [320. 321. 322. 323. 324. 325.]
 [330. 331. 332. 333. 334. 335.]
 [340. 341. 342. 343. 344. 345.]]
```

Matrix Multiplication

A Matrix is an array of numbers:

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}$$

A Matrix

(This one has 2 Rows and 3 Columns)

Matrix Multiplication

Multiplying a Matrix by Another Matrix

But to multiply a matrix **by another matrix** we need to do the "dot product" of rows and columns ... what does that mean? Let us see with an example:

To work out the answer for the **1st row** and **1st column**:

The diagram shows the calculation of the first element of the product matrix. It features two matrices: a 2x3 matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ and a 3x2 matrix $\begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$. A yellow curved arrow labeled "Dot Product" connects the first row of the first matrix (1, 2, 3) to the first column of the second matrix (7, 9, 11). The result of this dot product, 58, is shown in a yellow circle, which is the first element of the resulting product matrix $\begin{bmatrix} 58 & \dots \end{bmatrix}$.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & \dots \end{bmatrix}$$

Matrix Multiplication

The "Dot Product" is where we **multiply matching members**, then sum up:

$$(1, 2, 3) \bullet (7, 9, 11) = 1 \times 7 + 2 \times 9 + 3 \times 11 \\ = 58$$

We match the 1st members (1 and 7), multiply them, likewise for the 2nd members (2 and 9) and the 3rd members (3 and 11), and finally sum them up.

Matrix Multiplication

Want to see another example? Here it is for the 1st row and **2nd column**:

The diagram illustrates the calculation of the element in the first row and second column of the product matrix. It shows the first row of the first matrix, $[1 \ 2 \ 3]$, multiplied by the second column of the second matrix, $\begin{bmatrix} 8 \\ 10 \\ 12 \end{bmatrix}$. The result is the second element of the first row of the product matrix, 64 . Yellow highlights and arrows indicate the specific elements and the resulting value.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \end{bmatrix}$$

$$\begin{aligned} (1, 2, 3) \cdot (8, 10, 12) &= 1 \times 8 + 2 \times 10 + 3 \times 12 \\ &= 64 \end{aligned}$$

Matrix Multiplication

We can do the same thing for the **2nd row** and **1st column**:

$$(4, 5, 6) \cdot (7, 9, 11) = 4 \times 7 + 5 \times 9 + 6 \times 11 \\ = 139$$

And for the **2nd row** and **2nd column**:

$$(4, 5, 6) \cdot (8, 10, 12) = 4 \times 8 + 5 \times 10 + 6 \times 12 \\ = 154$$

And we get:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix} \checkmark$$

DONE!

Matrix Multiplication

In General:

To multiply an $m \times n$ matrix by an $n \times p$ matrix, the n s must be the same, and the result is an $m \times p$ matrix.

$$m \times n \times n \times p \rightarrow m \times p$$

Matrix Multiplication

```
1 array_1 = np.array([[1,2,3],[4,5,6]])  
2 array_2 = np.array([[7,8],[9,10],[11,12]])
```

```
1 print(array_1.shape)  
2 print(array_2.shape)
```

(2, 3)

(3, 2)

Matrix Multiplication

In General:

To multiply an $m \times n$ matrix by an $n \times p$ matrix, the n s must be the same, and the result is an $m \times p$ matrix.

$$m \times n \times n \times p \rightarrow m \times p$$

```
if array_1.shape[1] != array_2.shape[0]:  
    print("Matrix multiplication not possible, wrong dimensions")
```

Matrix Multiplication

$$m \times n \times n \times p \rightarrow m \times p$$

```
m = array_1.shape[0]
n = array_1.shape[1]
p = array_2.shape[1]

result = np.zeros((m,p))
```

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix} = \begin{bmatrix} \bullet \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet \\ & \bullet & \bullet \\ & & \bullet & \bullet \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet \\ & & \\ & & \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ & & & \\ & & & \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet \end{bmatrix}$$

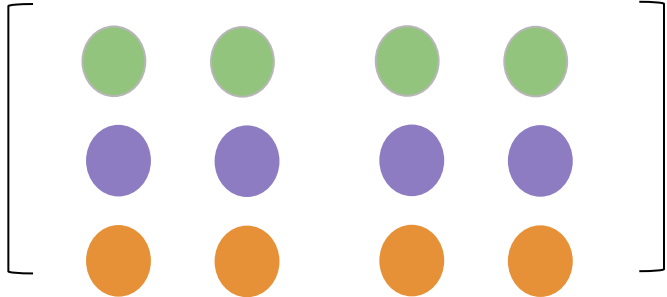
Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=



Matrix Multiplication

m = 3
p = 4
n = 3

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

= $\begin{bmatrix} \bullet = 1 \times 2 + 2 \times 6 + 1 \times 1 \end{bmatrix}$

Matrix Multiplication

```
1 def matrix_multiplication(array_1,array_2):
2
3     if array_1.shape[1]!=array_2.shape[0]:
4         print("Matrix multiplication not possible, wrong dimensions")
5         return
6
7     m = array_1.shape[0]
8     n = array_1.shape[1]
9     p = array_2.shape[1]
10
11     result = np.zeros((m,p))
12
13     for i in range(m):
14         for j in range(p):
15             sum=0
16             for k in range(n):
17                 sum += array_1[i][k]*array_2[k][j]
18             result[i][j] = sum
19
20     return result
21
```

Matrix Multiplication

```
1 def matrix_multiplication(array_1,array_2):
2
3     if array_1.shape[1]!=array_2.shape[0]:
4         print("Matrix multiplication not possible, wrong dimensions")
5         return
6
7     m = array_1.shape[0]
8     n = array_1.shape[1]
9     p = array_2.shape[1]
10
11     result = np.zeros((m,p))
12
13     for i in range(m):
14         for j in range(p):
15             sum=0
16             for k in range(n):
17                 sum += array_1[i][k]*array_2[k][j]
18             result[i][j] = sum
19
20     return result
21
```

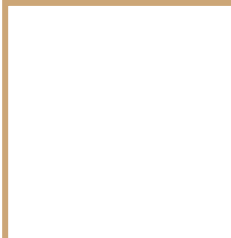
```
1 array_1 = np.array([[1,2,3],[4,5,6]])
2 array_2 = np.array([[7,8],[9,10],[11,12]])
```

```
1 print(array_1.shape)
2 print(array_2.shape)
```

```
(2, 3)
(3, 2)
```

```
1 print(matrix_multiplication(array_1,array_2))
```

```
[[ 58.  64.]
 [139. 154.]]
```



Linked List



Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!

Limitations of Array

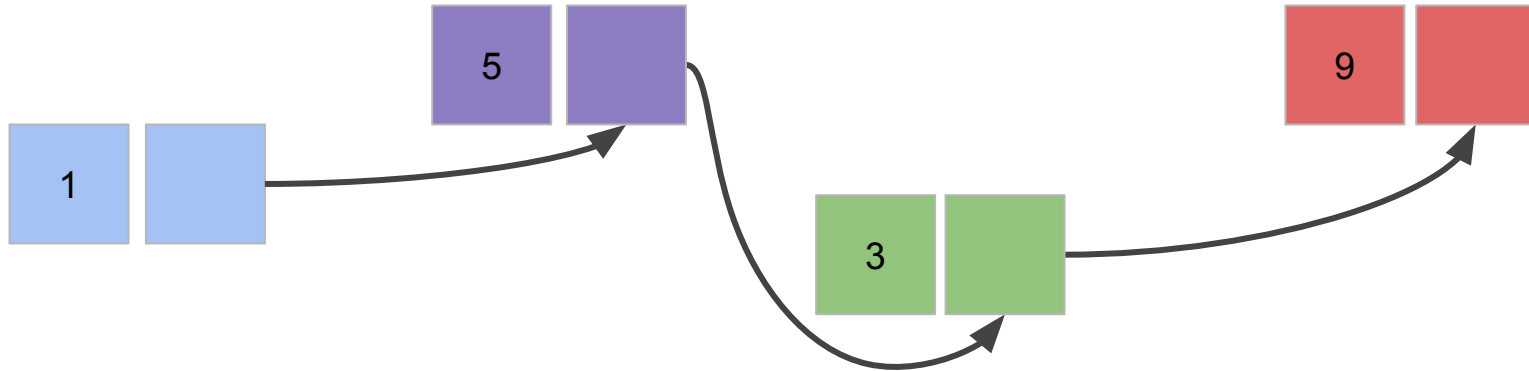
- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!
- **Shifting elements in removal** : Removing may also require shifting to plug the hole left by the removed element. If the ordering of the elements does not matter, we can avoid shifting by replacing the removed element with the element in the last slot (and then treating the last slot as empty).

Array vs. Linked List

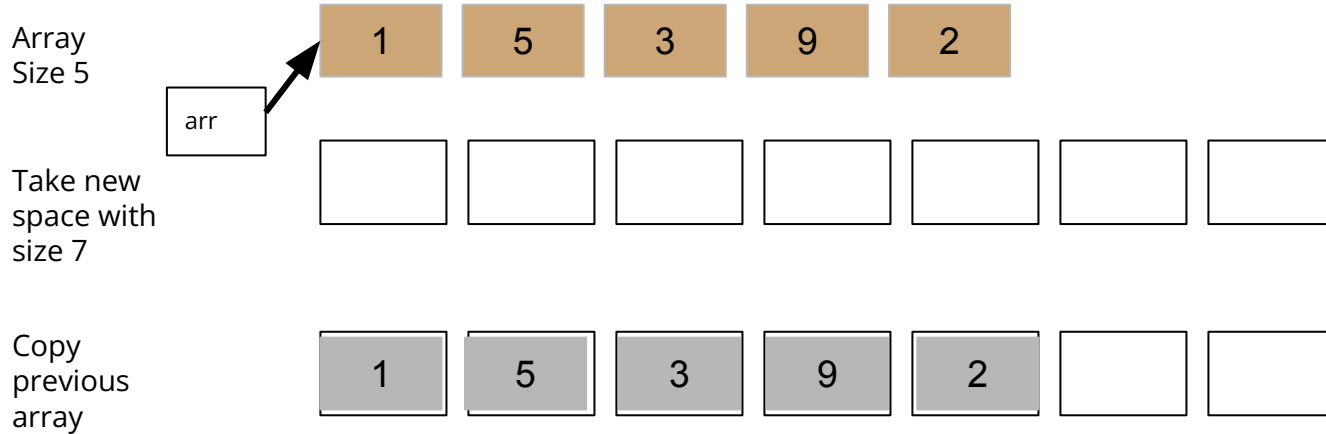
Array
in memory



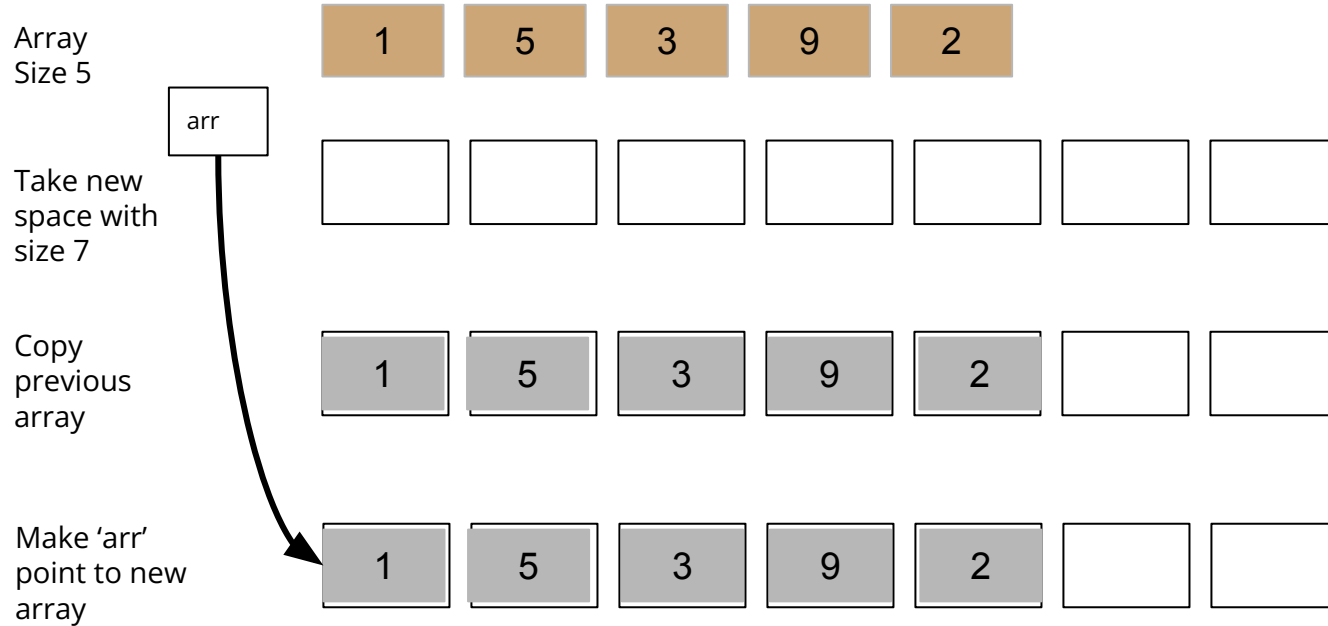
Linked list
in memory



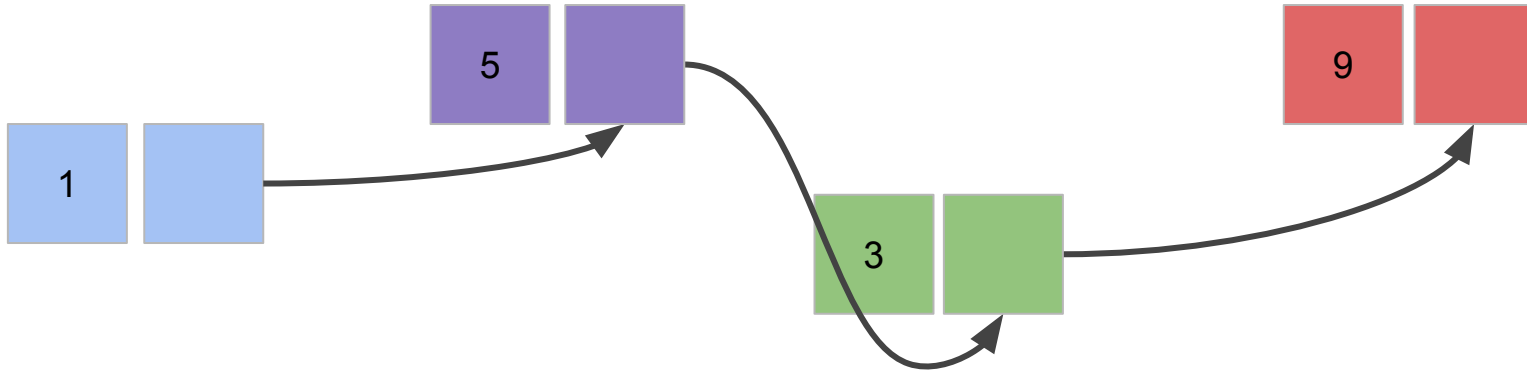
Resizing - Array



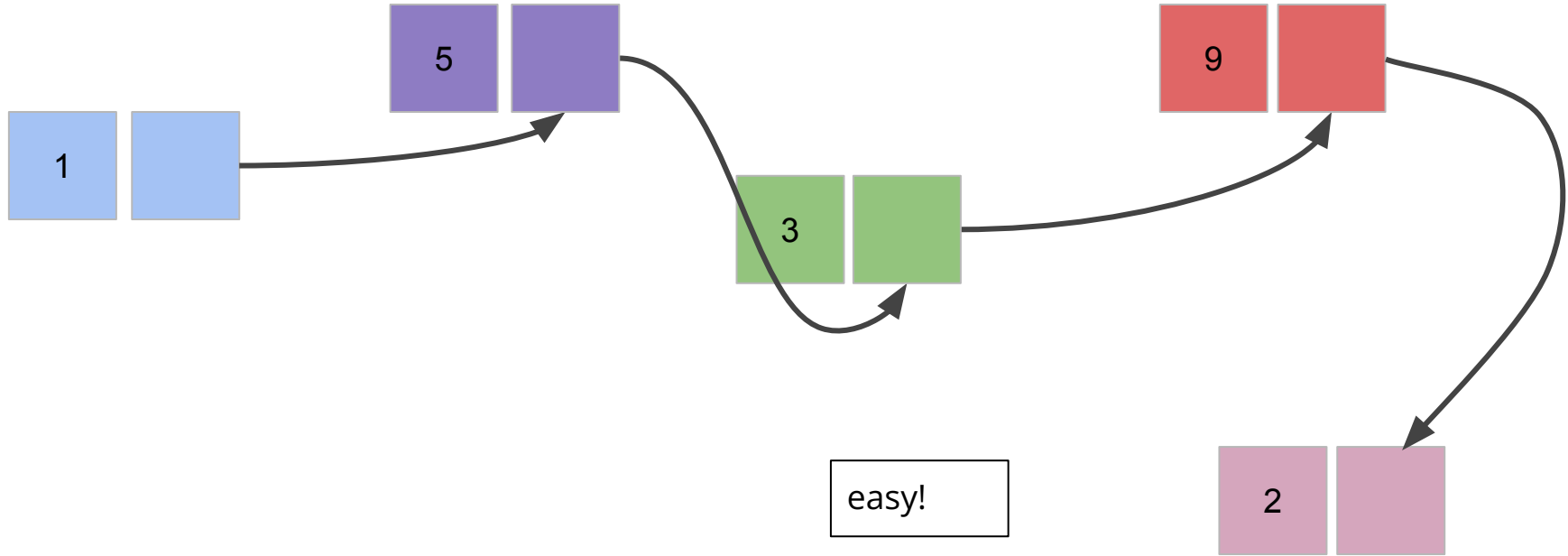
Resizing - Array



Resizing - Linked List



Resizing - Linked List



Insertion - Array

Array
Capacity 6
Size 5



Insert 7 here

Insertion - Array

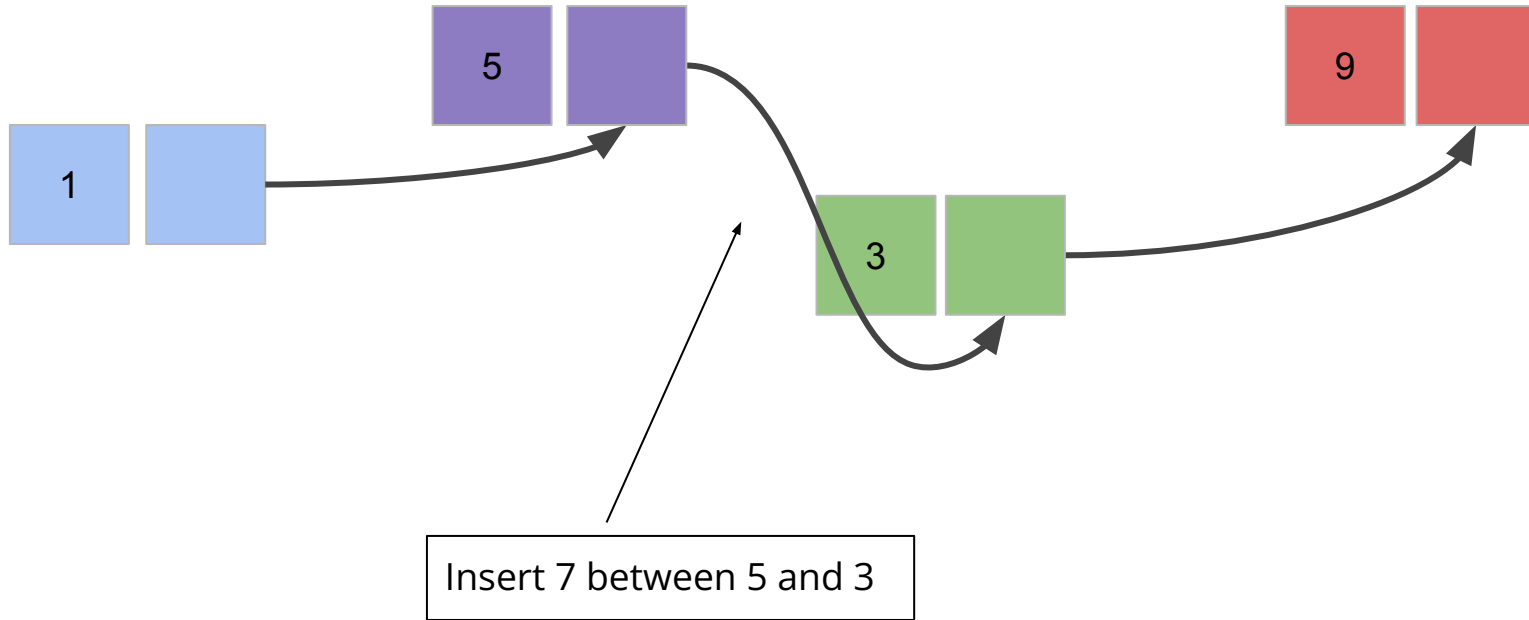
Array
Capacity 6
Size 5



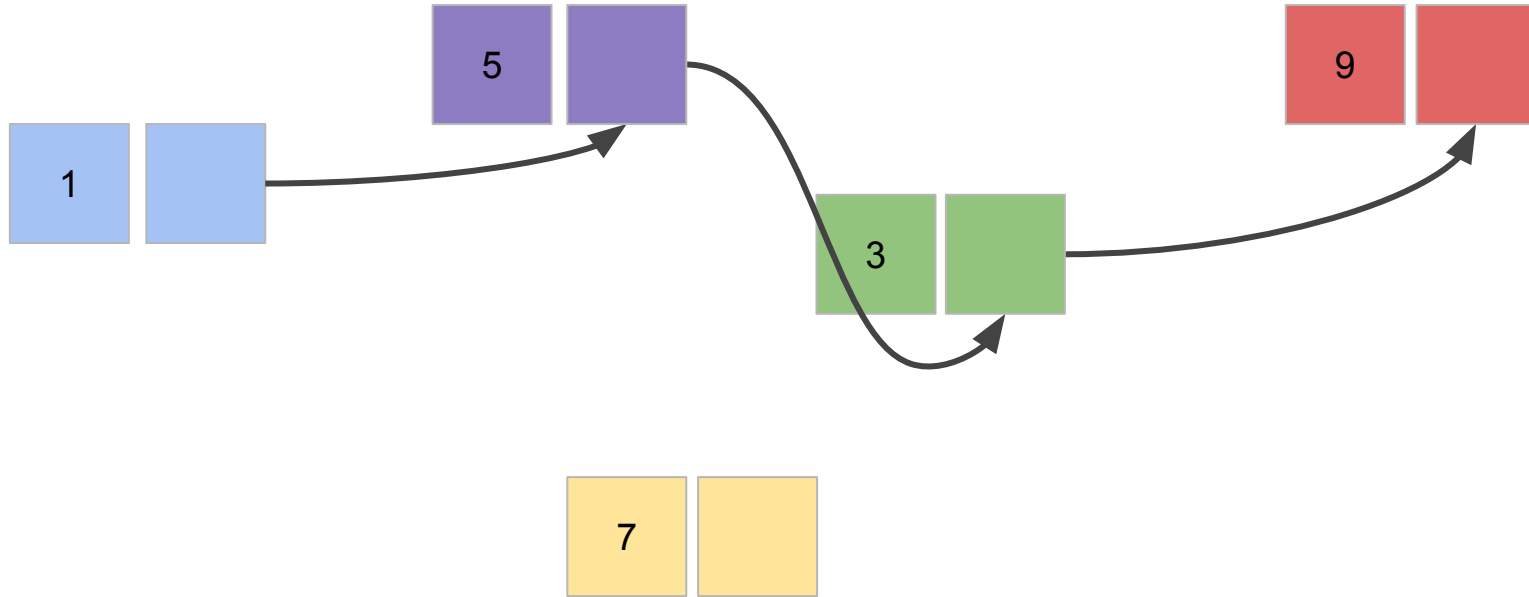
Insert 7 here

`array[3] = 7`

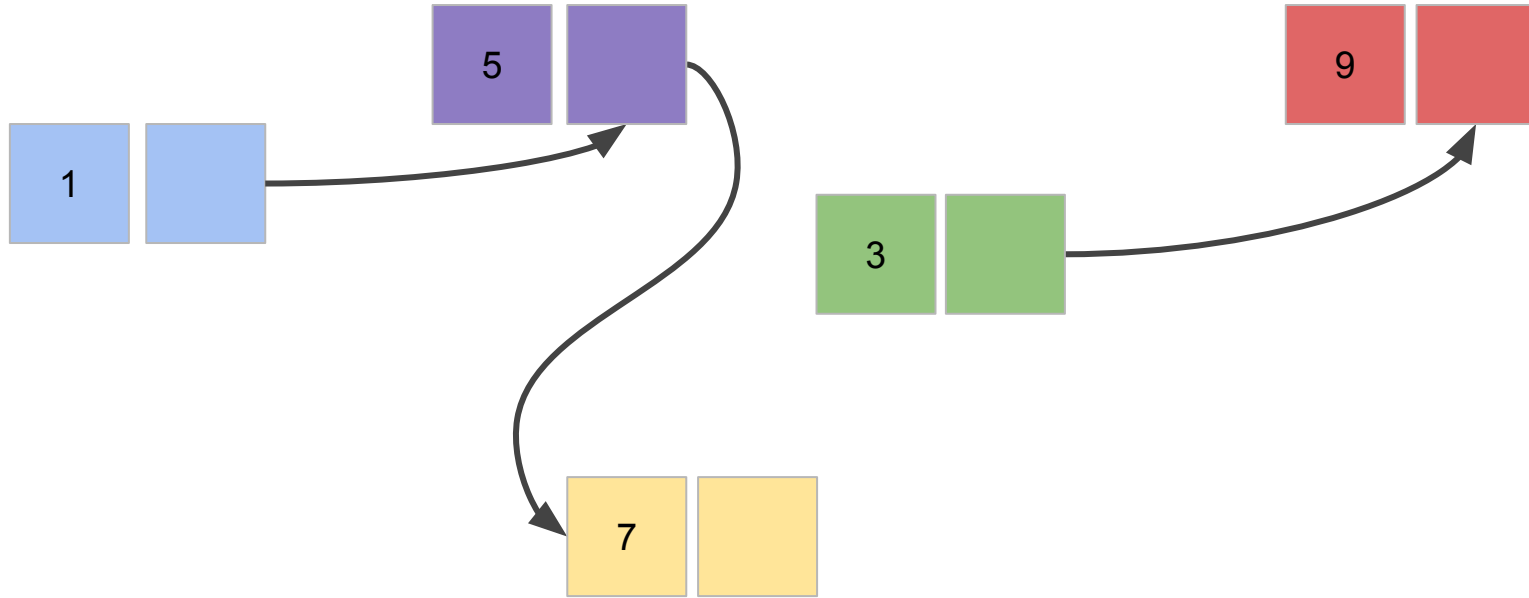
Insertion - Linked List



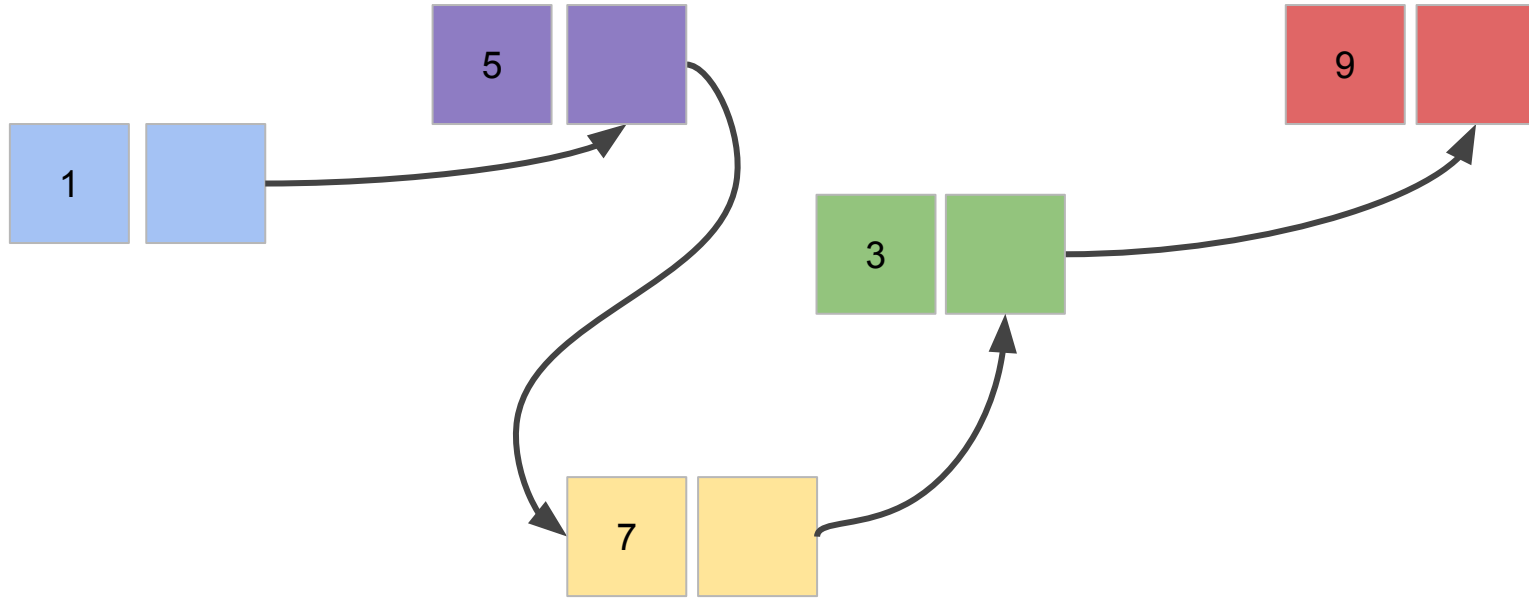
Insertion - Linked List



Insertion - Linked List



Insertion - Linked List



Removal - Array

Array



Delete 5

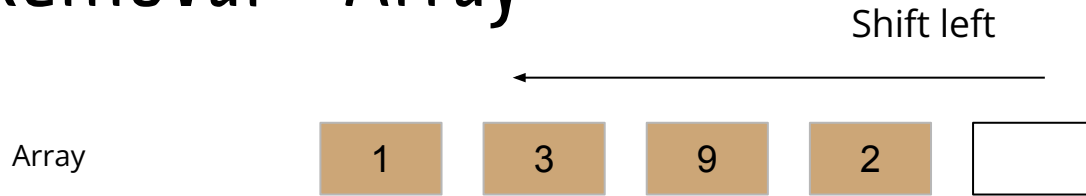
Removal - Array

Array



```
array[1] = None
```

Removal - Array



```
array[1] = None
```

Removal - Array

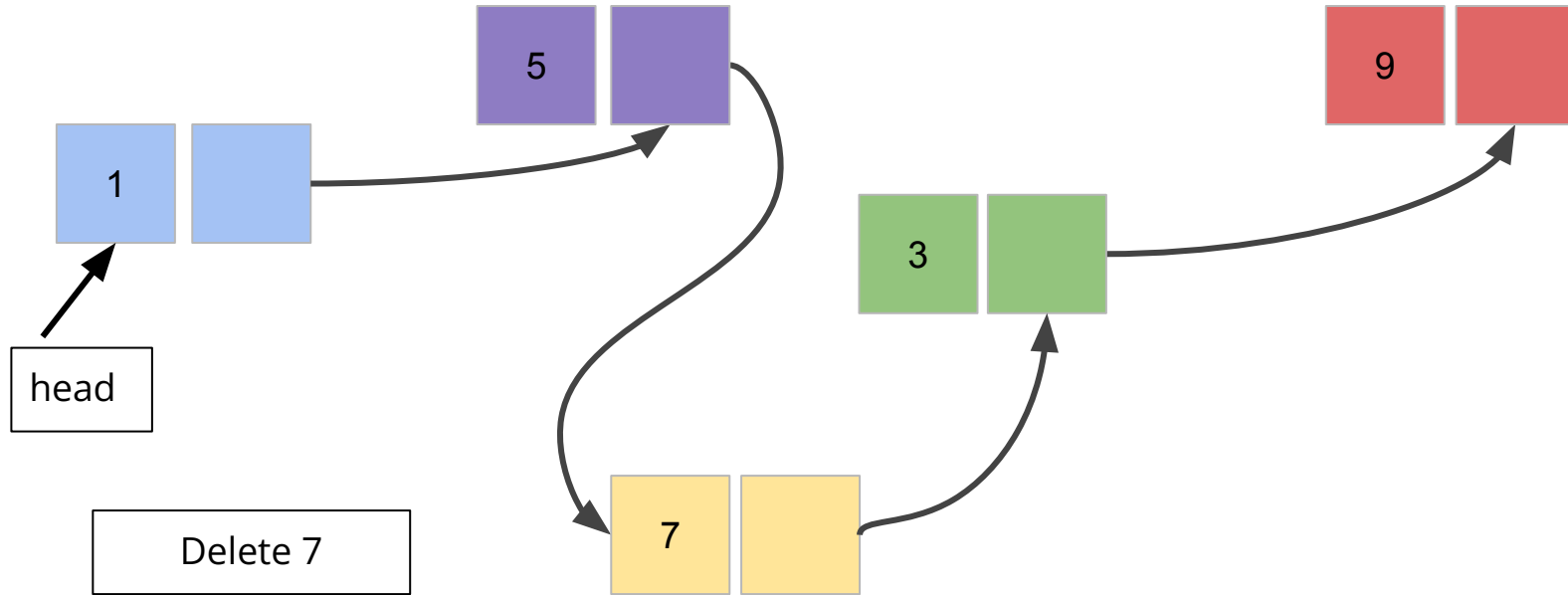
Array
Capacity 6
Size 5



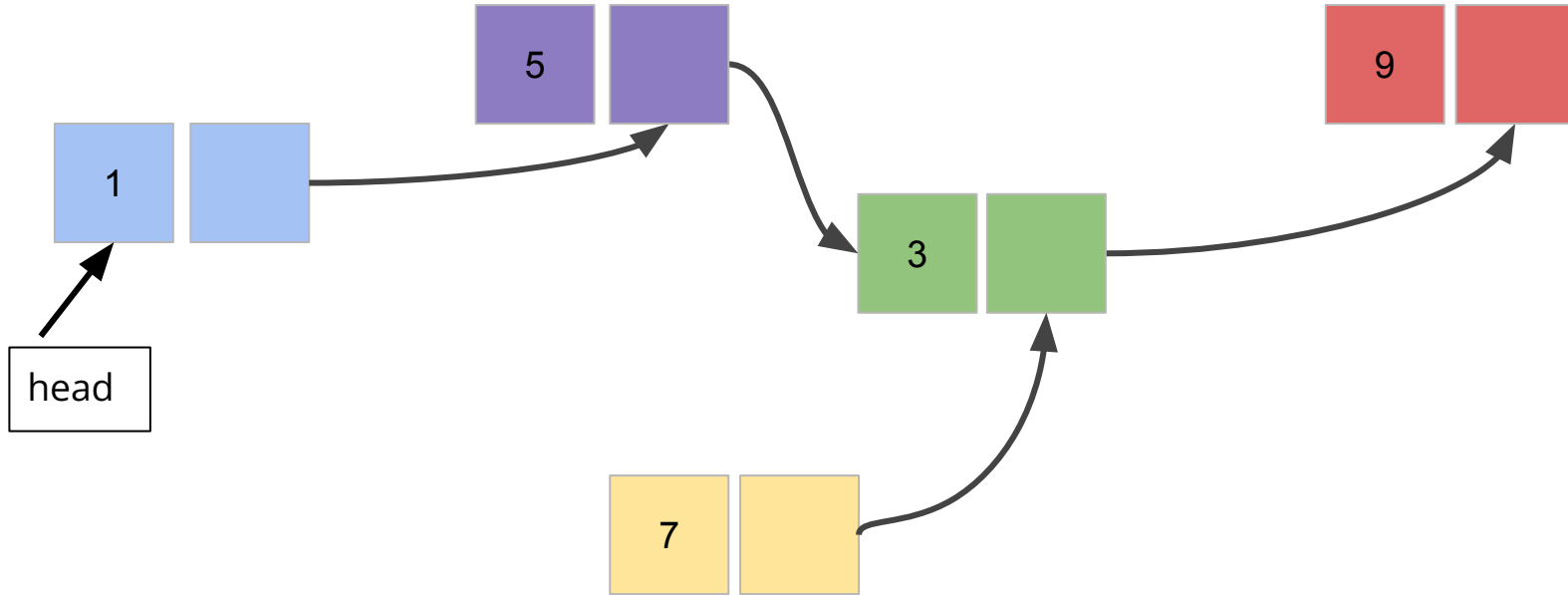
Insert 7 here

`array[3] = 7`

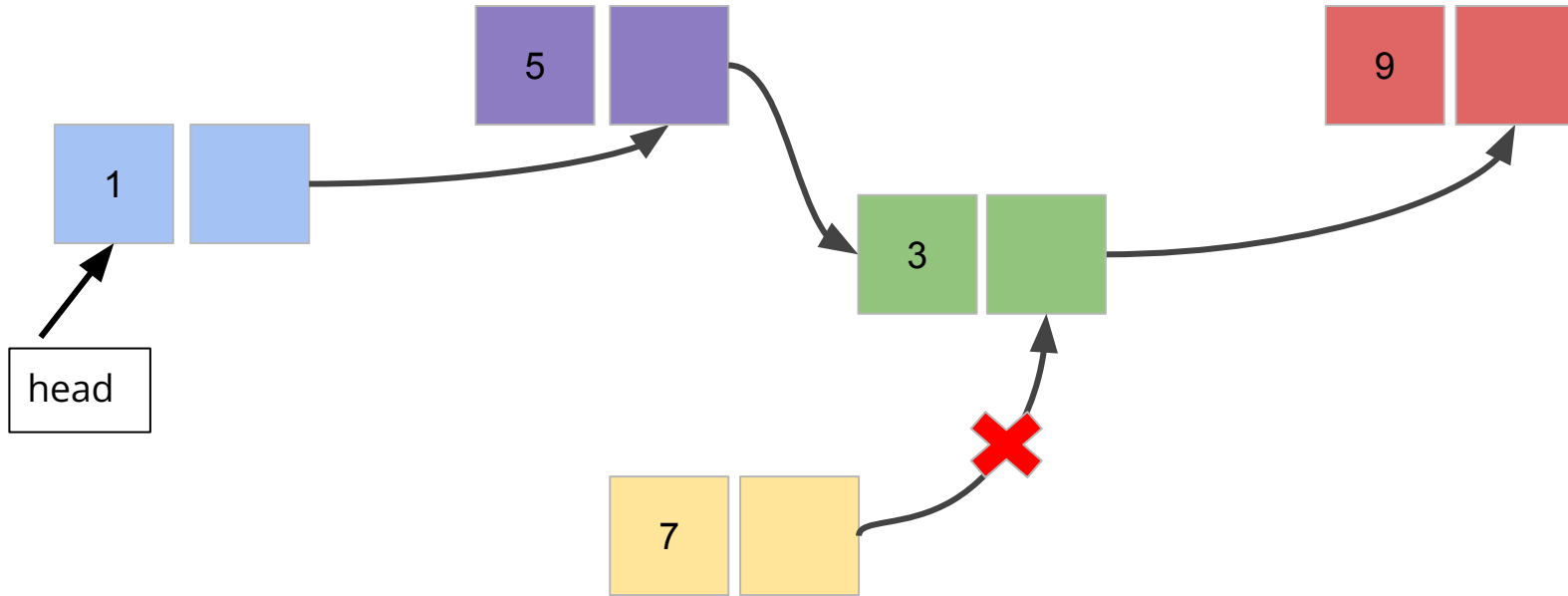
Removal - Linked List



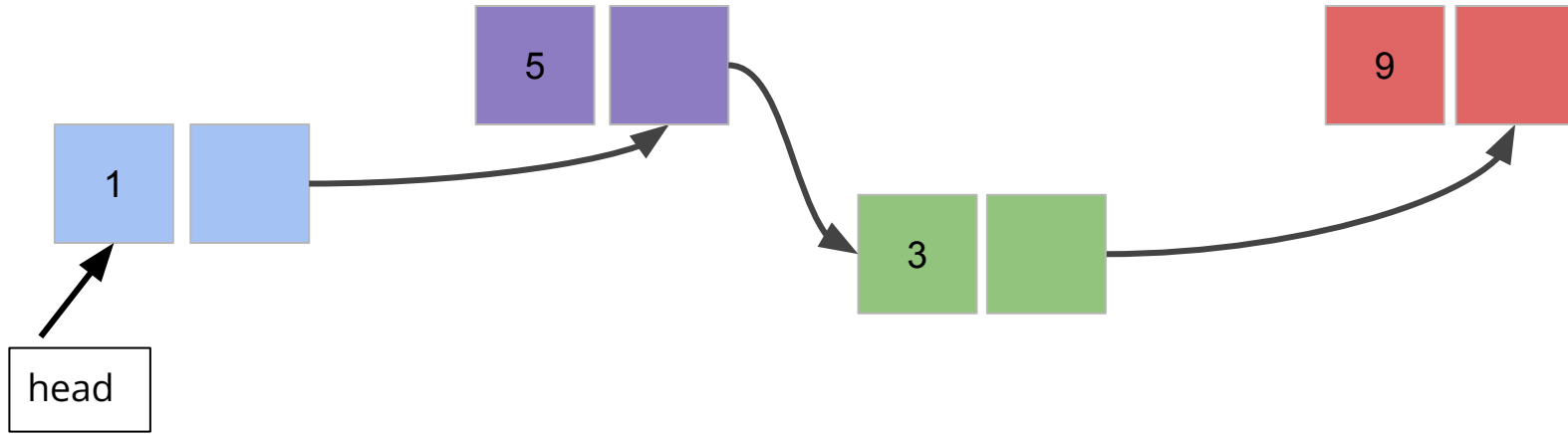
Removal - Linked List



Removal - Linked List

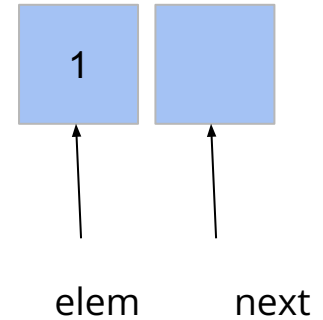


Removal - Linked List



Linked List Creation

```
1 # Node class design
2 class Node:
3     def __init__(self, e, n):
4         self.elem = e
5         self.next = n      (Address of next node)
6
```



Linked List Creation from an Array

```
1 # Creating a list
2 def createList(a):
3     head = Node(a[0], None)
4     tail = head
5     for i in range(1, len(a)):
6         n = Node(a[i], None)
7         tail.next = n
8         tail = tail.next
9     return head
```

Iterating Over a Linked List

```
1 # Iteration over a linked list
2 def iteration(head):
3     temp = head
4     while temp != None:
5         print(temp.element)
6         temp = temp.next
```

Counting elements of a Linked List

```
1 # Counting number of element in the list
2 def count(head):
3     count = 0
4     temp = head
5     while temp != None:
6         count += 1
7         temp = temp.next
8     return count
```

Retrieving an element from an index



```
1 # Getting element of an specific index
2 def elemAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp.element
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```


Retrieving a node from an index

```
1 # Getting node of an specific index
2 def nodeAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Update value of specific index

```
1 # Setting new element of an specific index
2 def set(head, idx, elem):
3     count = 0
4     temp = head
5     isUpdated = False
6     while temp != None:
7         if count == idx:
8             temp.elem = elem
9             isUpdated = True
10            break
11        temp = temp.next
12        count += 1
13    if isUpdated:
14        print("Value successfully updated!!!!")
15    else:
16        print("Invalid index")
```

Searching an element in the list

```
1 def contains(head, elem):  
2     temp = head  
3     while temp != None:  
4         if elem == temp.elem:  
5             return True  
6         temp = temp.next  
7     return False
```

Searching an element in the list

```
1 # Getting index of an specific element
2 def indexOf(head, elem):
3     temp = head
4     count = 0
5     while temp != None:
6         if elem == temp.elem:
7             return count
8         count += 1
9         temp = temp.next
10    return -1 # Here -1 represents the absence of element in the list
```

Inserting an element into a list

```
1 def insert(head, elem, idx):
2     total_nodes = count(head)
3     if idx == 0: # Inserting at the beginning
4         n = Node(elem, head)
5         head = n
6     elif idx >= 1 and idx < total_nodes: # Inserting at the middle
7         n = Node(elem, head)
8         n1 = nodeAt(head, idx - 1)
9         n2 = nodeAt(head, idx)
10        n.next = n2
11        n1.next = n
12    elif idx == total_nodes: # Inserting at the end
13        n = Node(elem, None)
14        n1 = nodeAt(head, total_nodes - 1)
15        n1.next = n
16    else:
17        print("Invalid Index")
18    return head
```

Removing an element from a list

```
1 def remove(head, idx):
2     if idx == 0: # Removing first element
3         head = head.next
4     elif idx >= 1 and idx < count(head): # Removing middle element
5         n1 = nodeAt(head, idx - 1)
6         removed_node = n1.next
7         n1.next = removed_node.next
8     else:
9         print("Invalid Index")
10    return head
```